

Graduation Project: Generative AI Software Product

Design and develop a complete AI-powered software solution for a real-world problem.

1. Project Objective

Each group must design and develop a **Generative AI software product** that solves a real-world problem for a clearly defined **target audience**.

Each group is responsible for selecting its own:

- Real-world problem and application domain.
- Target audience and user needs.
- Knowledge base and data sources.
- Suitable open-source LLM and AI models.

The project must not be a general-purpose chatbot. The AI assistant must have a **clear objective** and provide measurable value to its intended users.

Possible domains include education, finance, tourism, engineering, agriculture, legal information, customer support, accessibility, career development, and other suitable non-medical domains.

2. Main Product Requirement

The main feature of the application must be an **AI conversational assistant** supporting:

- **Text interaction**
- **Voice interaction using Speech-to-Text (STT)**

Text-to-Speech (TTS) for voice responses is optional but recommended.

3. Required AI Pipeline

Voice/Text → STT → Prompt Engineering → RAG → Fine-Tuned & Optimized LLM → Response → Optional TTS

The project must include:

- **Speech-to-Text:** Use Whisper or an equivalent multilingual STT model.
- **Prompt Engineering:** Apply suitable techniques such as system prompting, few-shot prompting, structured prompting, context-aware prompting, or reasoning-based prompting.
- **Retrieval-Augmented Generation (RAG):** Build a domain-specific knowledge base using sources such as PDFs, books, manuals, reports, FAQs, documents, or structured datasets.
- **LLM Fine-Tuning:** Fine-tune an open-source LLM using a dataset related to the selected problem. Compare the **Base LLM vs. Fine-Tuned LLM**.
- **LLM Optimization:** Apply at least one optimization technique such as **LoRA, QLoRA, 4-bit/8-bit Quantization, or Model Compression**. Analyze its effect on model size, memory usage, inference speed, and response quality.

4. Software Requirements

Frontend:	React
Backend:	FastAPI
AI/ML:	Python
LLM:	Open-source Large Language Model
RAG:	LangChain or equivalent
Vector Database:	FAISS, ChromaDB, Qdrant, or equivalent
STT:	Whisper or equivalent
TTS:	Optional but recommended

The final system must integrate the frontend, backend, RAG pipeline, AI models, and voice features into a complete working software product.

5. Minimum Required Features

- Clearly defined problem, target audience, and chatbot objective.
- Text and voice-based interaction.
- Speech-to-Text integration.
- Domain-specific RAG knowledge base.
- Prompt Engineering techniques.
- Fine-tuned open-source LLM.
- Base vs. Fine-Tuned model comparison.
- At least one LLM optimization technique.
- Conversation history/context.
- React frontend and FastAPI backend.
- Complete end-to-end system integration.

6. Deliverables

1. Documentation

A project report covering the problem statement, target audience, solution, system architecture, dataset/knowledge base, RAG, Prompt Engineering, fine-tuning, optimization, model evaluation, results, limitations, and future improvements.

2. GitHub Repository

A complete and organized repository containing the React frontend, FastAPI backend, AI/RAG code, fine-tuning and optimization code/notebooks, dependencies, setup instructions, and README.

3. Working Demo

An end-to-end demonstration showing the real-world use case, text interaction, voice/STT interaction, RAG retrieval, LLM responses, fine-tuned model behavior, and React-FastAPI integration.

Final Goal: Build a real AI software product, not only a model, notebook, or general-purpose chatbot.